**Technical Architecture & Operations Manual**

# Alza Customer Support Bot Service

**GCP Project**: `gen-lang-client-0958735221`

October 2026

**Abstract**

This document describes the architecture, development and deployment of the **Alza Case Study - Email-based AI chatbot** assignment.

## Contents

**1 Assignment description** **2**

**2 Work break-down structure** **2**

**3 Prerequisites** **3**

**4 Development** **3**
- 4.1 Proof of concept app . . . 3
- 4.2 Attachement processing . . . 3
- 4.3 Advanced function implementation - Option B: Retrieval-Augmented Generation (RAG) . . . 4
- 4.4 Architectural Shift: Polling vs. Pub/Sub routines . . . 5
  - 4.4.1 Automated Sender & Loop Mitigation . . . 7
  - 4.4.2 Idempotent State Locking & Label-Based Deduplication . . . 7

**5 Architecture & Codebase Structure** **8**
- 5.1 Unit Testing & Quality Assurance . . . 9
- 5.2 GCP Infrastructure Specification . . . 9
- 5.3 Final architecture and system flow diagram . . . 10

**6 Gaps and potential enhancements** **11**

**7 Appendix** **12**
- 7.1 Full codebase structure tree . . . 12
- 7.2 Running app screenshots . . . 13
- 7.3 Conversation examples . . . 14

# 1 Assignment description

The objective was to create a proof-of-concept for an intelligent assistant that can autonomously handle email inquiries. The chatbot should read incoming emails from a designated email inbox, understand their context and content (including attachments, at minimum **PDFs**, **audio files** and **images**), generate a relevant and intelligent response using a Large Language Model (LLM), and reply within the original email thread.

The entire process should take no more than **2 minutes** from the point of receiving to sending a reply. The solution also needs to be built and deployed on a public cloud platform.

# 2 Work break-down structure

To fully understand the task requirements and set up clear goals for delivery, a simple work breakdown structure was created:

- Prerequisites
  - Create new email
  - Create new GCP project [1]
  - Obtain a free Gemini API key
- Local development
  - Set up local dev environment
  - Create an email inbox reader and sender
  - Build an LLM-based email processor
  - Add one advanced feature
  - Test solution locally
- Cloud development
  - Dockerize the locally developed app
  - Upload the container to GCP
  - Set up all permissions
  - Validate operability and functionality
- Finalisation
  - Codebase cleanup
  - Final review
  - Documentation write-up
  - Submission

[1] Google Cloud Platform was selected based on previous experience, serverless deployment options and simplified Gmail and Gemini integration.

# 3 Prerequisites

First, a new Gmail account (alza.case.study@gmail.com) was made solely for the purpose of this assignment. This account was then used to create a free-tier project in GCP and to obtain a free Gemini API key from Google AI Studio (a new Gemini key from GCP in the free-tier project was throwing resource exhaustion errors, even though there were plenty of initial free credits left)

# 4 Development

## 4.1 Proof of concept app

To develop the solution, Python was selected as the go-to programming language due to the availability of many libraries for AI-based applications, fast development and no front-end requirements.

Initially, a simple three-component architecture was created as a proof-of-concept: a **main.py** script running the main project logic loop (get email -> process it with Gemini -> send response), a **gmail_handler.py** that was able to connect to Gmail with the specified account credentials to read and send emails, and a **gemini_handler.py** to send and receive messages from Gemini.

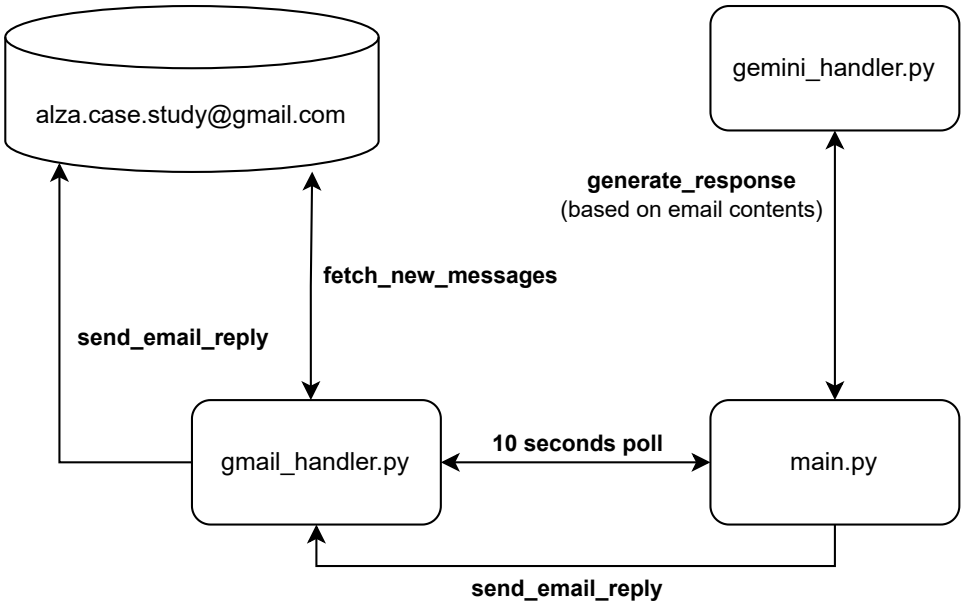



Figure 1: Initial PoC architecture

To allow the app to access email, two-factor authentication needed to be enabled on the given Gmail account to obtain an App Password for Gmail IMAP via an SSL connection. The inbox was polled in an infinite (while=True) loop every 10 seconds to see if there were any new unprocessed messages (status, messages = mail.search(None, 'UNSEEN')). The email body and any attachments were collected and sent to Gemini via the API using a simple system prompt requesting a response to the query. The response was converted into an email in response to the original message (based on its *message_id*) and sent via SMTP from the support mailbox to the original sender.

## 4.2 Attachement processing

To better understand users and their requests, more detailed attachment processing is needed. In the initial version, the attachments were only understood and accessible to Gemini when

drafting a response, but to prepare for the addition of advanced functionality, this context comprehension needed to be performed before drafting the response.

Simple attachment validation steps were put in place: correct and supported file extensions check, file name sanitation (to prevent path traversals) and magic byte validation[2] (file type signature validation).

An additional LLM call was added for non-textual files (images and audio files) in case of missing or insufficient request context in the email body, utilising the multimodality and fast responsivity of Gemini models. Its response was then treated as the request body itself for intent classification (see next subsection 4.2)

### 4.3 Advanced function implementation - Option B: Retrieval-Augmented Generation (RAG)

The implementation of RAG-enhanced responses was selected to provide more grounded responses from the agent when the request is related to a specific topic. For this, a set of three documents containing information about a fictional Alza product - **AeroSpace Pulsar-1 Personal Orbital Shuttle Single-Occupant Sub-Orbital & Low-Earth-Orbit Class Vehicle** - were generated by Gemini:

- User diagnostics & manual.pdf
- Spare parts inventory.xlsx
- Control panel view.jpg

These documents are of different file types, in order to show that each of these files requires different steps of preprocessing (parsing and chunking):

- pdf document - Split by major section headings and error code descriptions
- xlsx document - Split by rows (spare parts entries)
- jpg image - Sent to Gemini to create a description, stored as one chunk

To make use of these chunks, they were transformed into embeddings with the **sentence-transformers** library, more specifically. the **all-MiniLM-L6-v2** embedding model. From these embeddings, two index files were built - one dense Faiss index for vector retrieval and one sparse BM2 index for keyword search. These two indices were selected in order to allow hybrid search upon the knowledge base [3]

To enhance the email response creation request for Gemini, a **rag_handler.py** script was created, whose main job consists of two steps: classifying the incoming request and adding relevant context if necessary. The classification of the message was first done in a quick keyword lookup (any email body matching "ERR-", "PLS-", "SKU", "PRICE", "BATTERY", "SCRUBBER", "SHIELD", or "NOZZLE") and, in case of failure, a Gemini-assisted classification[4]. If the keyword match or LLM classification was successful, the hybrid search began by vectorising and token-splitting the incoming query. The final scores of retrieved chunks were computed as follows:

[2] https://en.wikipedia.org/wiki/List_of_file_signatures

[3] This might be an overkill for just three documents, but the same principle is scalable across many more documents, where this approach will provide faster and better quality retrieval.

[4] Initially a fully offline 'all-MiniLM-L6-v2' / FAISS indexing stack was used, which bloated the full app docker image up to 6GB. To slim down the size and speed up deployment, it was replaced by Gemini API embedding calls paired with lightweight NumPy vector search- document chunks are pre-embedded using Gemini's API and saved as flat matrix files ('.npy'). At runtime, the agent computes exact cosine similarity via vectorized NumPy matrix dot-product operations, which execute in milliseconds for small-to-medium knowledge bases.

combined_scores[idx] = (0.6 * faiss_score) + (0.4 * bm25_score)

The top 3 highest-scoring chunks were then injected as additional context for Gemini, alongside the original email body and its attachments.

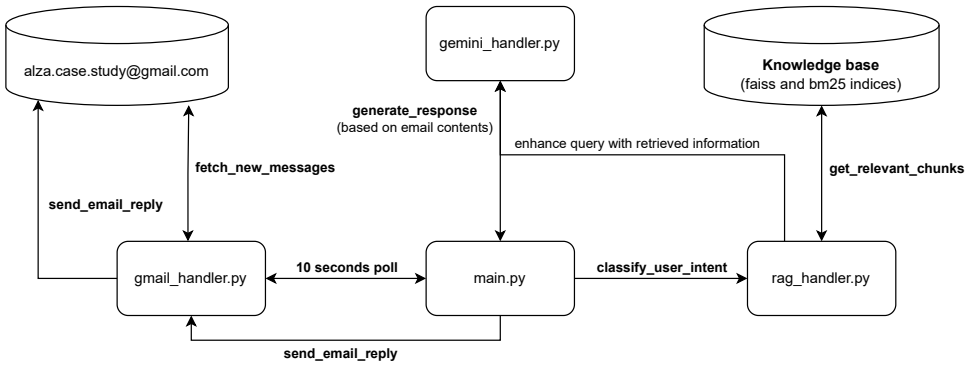



Figure 2: RAG enhanced architecture

### 4.4 Architectural Shift: Polling vs. Pub/Sub routines

The 10-second polling interval pattern was deemed very wasteful in consumption, API quota depletion, and friction with serverless scaling primitives. Therefore, a new version more suitable for GCP Cloud Run was developed - an **Event-Driven Push Model** utilising Gmail API and Pub/Sub methods, enclosed in a lightweight Flask application with a *health_check* and */pubsub/push* endpoints:

1. **Topic creation**: In GCP, a new Pub/Sub topic (an entry point where message producers (publishers) send data, and from which message consumers (subscribers) receive messages) was created.
2. **Ingestion**: Gmail receives a new customer message and triggers a notification in the topic.
3. **Pub/Sub Pipeline**: Google Cloud Pub/Sub receives the event on the topic.
4. **Push Webhook**: Pub/Sub delivers an HTTP POST request to the Cloud Run endpoint (`/pubsub/push`).
5. **Execution**: Cloud Run scales from zero, processes the message via Gemini API, sends the automated response, and returns HTTP 200.

To create a publisher, the Gmail API had to be enabled first, and a *watch* on the inbox was registered by running a helper script (located in the repository under the *google/* folder), which required a client secret and a refresh token - these were obtained by creating an OAuth 2.0 desktop client of this project in GCP. This client allows Google's internal Gmail service to automatically publish the message to the *pub/sub* topic (including *read* permission on the inbox, registered as a **Test User** in the GCP project) and authorises the inbox handler part of our application to send responses (including message tagging).

The watch has a default 7-day expiration lease, so the renewal was automated by creating a Cloud Scheduler job that periodically calls the */admin/renew-watch* endpoint that sends a request to Gmail to keep publishing messages to the specified topic on behalf of the user.

For local development (WSL), a small workaround had to be created: to expose the endpoint, a free **ngrok**[5] subscription was created, which provided a public URL for exposing the endpoint by rerouting the incoming traffic to *http* port 8080 on localhost when

[5] ngrok.com

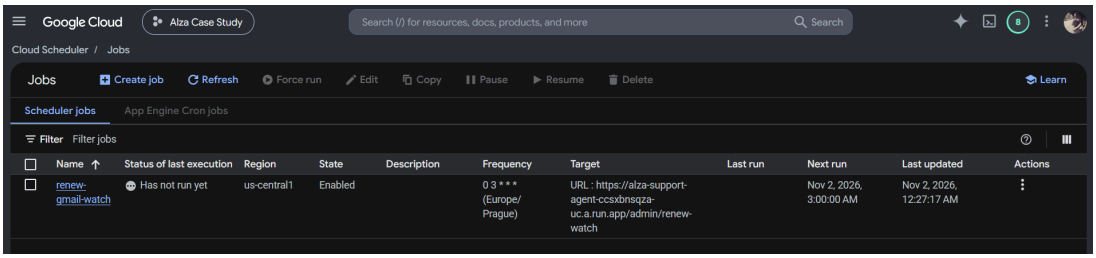



Figure 3: Cloud scheduler view in GCP - the "0 3 * * *" frequency means that the refresh runs at 3:00 AM every single day.

ngrok http 80 –url https://tall-bronco-captive.ngrok-free.dev

was run.

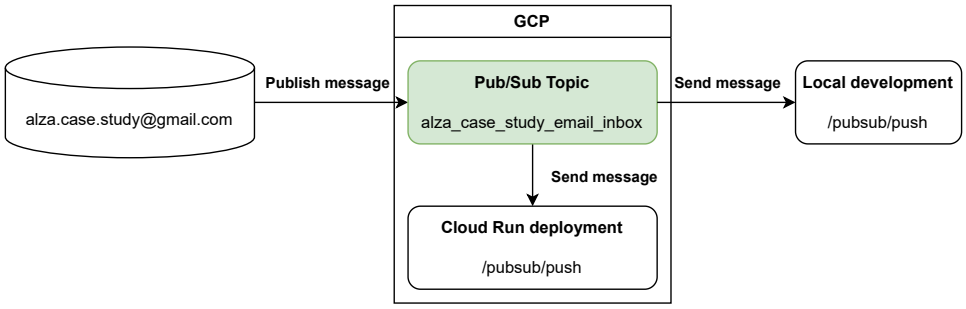



Figure 4: Pub/Sub simplified architecture with one publisher (Gmail inbox) and two subscribers (**local development app** and **GCP Cloud Run deployment**)

When a message arrives, Pub/Sub encapsulates metadata into a base64-encoded structure. The webhook endpoint unwraps this envelope to obtain the `emailAddress` and `historyId`.

| Field | Source | Purpose |
|---|---|---|
| `message.data` | Pub/Sub Payload | Base64 JSON containing event metadata |
| `emailAddress` | Decoded Data | Target mailbox receiving the support request |
| `historyId` | Decoded Data | Marker to fetch exact incoming message details |

Table 1: Structure of the event notification payload.

To ensure system stability, prevent infinite auto-reply cascades, and eliminate race conditions caused by concurrent event triggers, two core security subsystems were implemented: **Automated Sender & Loop Mitigation** and **Idempotent State Locking**.

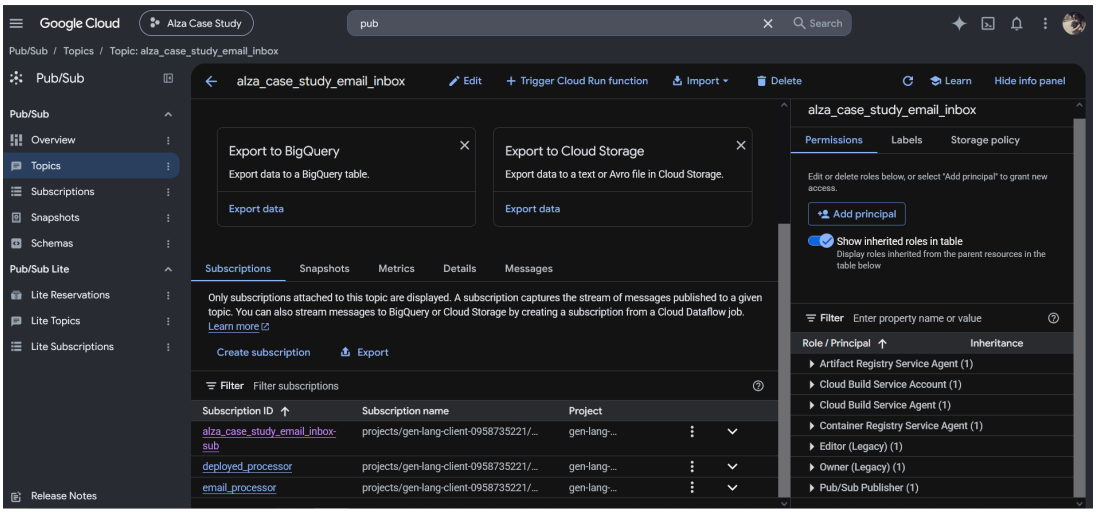



Figure 5: Pub/Sub middleware view in GCP. The *alza_case_study_email_inbox* has two subscribers: local development and Cloud Run app push endpoints.

### 4.4.1 Automated Sender & Loop Mitigation

A message is flagged as automated if it triggers any of the following checks:

1. **Self-Address Matching:** The extracted envelope sender address (`Reply-To` or `From`) is compared against the system's active support mailbox (`support_address`). If a message originates from the bot's own address, processing halts immediately.

2. **Automated Address Prefix Analysis:** The sender's address string is evaluated against a pre-compiled set of system prefixes ($\mathcal{P}$):

$$\mathcal{P} = \{\text{“noreply”}, \text{“no-reply”}, \text{“donotreply”}, \text{“mailer-daemon”}, \text{“postmaster”}\}$$

   If any $p \in \mathcal{P}$ is detected within the address substring, the email is categorised as non-interactive system traffic.

3. **RFC Header Inspection:** Standardised email headers are parsed for machine-generated signals:

   - `Auto-Submitted`: Verified to confirm the value equals `"no"`. Any other value (such as `"auto-generated"` or `"auto-replied"`) triggers immediate rejection.
   - `Precedence`: Rejects messages where the header field matches `"bulk"`, `"list"`, or `"junk"`.
   - `List-Unsubscribe`: The existence of this header explicitly identifies mass distribution or newsletter traffic, terminating execution prior to LLM evaluation.

### 4.4.2 Idempotent State Locking & Label-Based Deduplication

Google Cloud Pub/Sub guarantees *at-least-once* delivery semantics. Under heavy load or minor network latency, the service may dispatch multiple HTTP push notifications for a single mailbox event. Without proper deduplication and concurrency control, multiple server workers could process the exact same message simultaneously, generating duplicate responses or corrupted drafts.

The application addresses this through a two-tier control architecture combining local thread locks with explicit Gmail label state transitions:

- **Thread-Level Synchronization (`threading.Lock`):** A mutual exclusion lock (`_lock`) wraps the batch execution context in `process_pending()`. This forces concurrent webhook pushes hitting the same worker container to serialise execution, processing pending emails in strict sequential batches.
- **Label-Exclusion Querying:** When fetching unhandled messages from the Gmail API, the application executes a strict negative search filter:

  $$\text{Query} = \texttt{in:inbox newer_than:2d -label:alza-ai-processed}$$

  By specifically filtering out the custom `alza-ai-processed` label, any email that has completed the handler lifecycle is rendered completely invisible to subsequent polling loops.
- **Atomic Processing State Lifecycle:** The state transition sequence is structured to preserve fault tolerance:

  1. Fetch unlabelled email from Gmail API.
  2. Execute intent classification, knowledge retrieval, and draft/reply generation.
  3. Dispatch reply or commit draft to Gmail store.
  4. **Final Step:** Apply `alza-ai-processed` label to the email object.

  If a failure occurs during steps 1 through 3, the final labelling step is intentionally skipped. The message remains unlabelled, allowing Pub/Sub to safely retry execution without risking inconsistent intermediate states.

# 5 Architecture & Codebase Structure

After the PoC phase was completed, the full codebase was modularised into distinct operational layers located within the `app/` package:

- `config.py`: Centralised runtime configuration loading environment variables from system environments or local `.env` files. Encapsulates model settings (`GEMINI_MODEL`), GCP credentials/Vertex AI flags, Gmail OAuth parameters, Pub/Sub topic/audience specs, and knowledge base vector dimensions.
- `models.py`: Defines domain data structures using Python dataclasses (`IncomingEmail`, `Attachment`, `Reply`, `Evidence`) and Pydantic models (`Classification`). Enforces structured schema outputs from Gemini, including intent category, language detection, and search necessity.
- `server.py`: Flask HTTP web server providing operational endpoints: `GET /health` for container probes, `POST /pubsub/push` for decoding Pub/Sub envelopes and triggering processing, and `POST /admin/renew-watch` for lease renewals. Validates Google-signed OIDC Bearer tokens when configured.
- `agent.py`: Implements the `SupportAgent` class managing connections to the Gemini API (via direct key or Vertex AI). Orchestrates zero-shot intent classification and grounded, polite response drafting while maintaining system prompt guardrails against prompt injection.
- `gmail_client.py`: Thread-safe interface wrapper over the Gmail API. Manages RFC-compliant MIME message building, OAuth refresh token authorisation, unread inbox querying, custom label application (`alza-ai-processed`), and 7-day push watch subscriptions.

- `knowledge_base.py`: In-memory hybrid retrieval system combining cosine similarity over normalised Gemini text embeddings (`embeddings.npy`) with BM25 keyword matching (`bm25.pkl`). Scores and ranks technical documentation chunks to ground LLM draft generation.
- `embeddings.py`: Encapsulates Gemini embedding API calls (`gemini-embedding-001`). Handles text chunk batching (up to 50 items per request) and performs L2-normalisation on output vectors to ensure vector dot products equal cosine similarity.
- `email_handler.py`: Core orchestration engine that processes pending inbox emails using thread locks. Performs anti-loop automated sender checks, triggers AI classification and knowledge retrieval, manages draft/send execution modes, and updates processing status in Gmail.
- `attachments.py`: In-memory attachment validation engine. Prevents path-traversal and malicious file execution by enforcing file extension whitelisting (`.pdf`, `.png`, `.jpg`, `.mp3`, etc.), individual/total size limits (10 MB / 18 MB), and magic-byte signature verification.

### 5.1 Unit Testing & Quality Assurance

The system includes a simple automated test suite implemented with `pytest`. Every test uses isolated mock objects and parameterisation without requiring active network connectivity or real GCP credentials.

- **Attachment Tests (`test_attachments.py`)**: Validates in-memory security filters, checking that whitelisted formats pass, spoofed extensions and binaries are rejected via magic-byte signatures, size limits are enforced, and path traversal strings are stripped.
- **Email Handler Tests (`test_email_handler.py`)**: Tests pipeline execution with mock agents and clients, confirming correct draft/send modes, knowledge base conditional routing, automated sender filtering, and resilient error handling for pending messages.
- **Server Tests (`test_server.py`)**: Exercises Flask endpoints via ‘test_client()‘, verifying health check statuses, Pub/Sub webhook handling, administrative watch renewals, and error propagation codes.

### 5.2 GCP Infrastructure Specification

- **GCP Project ID**: `gen-lang-client-0958735221`
- **Artifact Registry**: `us-central1-docker.pkg.dev/gen-lang-client-0958735221/alza-repo/alza-support-agent:latest`
- **Pub/Sub Topic**: `alza_case_study_email_inbox`
- **Pub/Sub Subscription**: `alza_case_study_email_sub`

The application runs in a Cloud Run environment; a Docker image was created locally from the *app/* modules and pushed to the Google Cloud project's Artefact Registry. The image was built on GCP by invoking Google Cloud Build on this artefact and deployed by running a *gcloud run deploy* command with these parameters:

```
  gcloud run deploy $SERVICE \
--image $IMAGE --region $REGION \
--service-account $RUN_SA \
--no-allow-unauthenticated \
```

```
--memory 512Mi --cpu 1 \
--min-instances 0 --max-instances 1 \
--timeout 300 \
--set-env-vars GEMINI_MODEL=gemini-3.5-flash,
SUPPORT_ADDRESS=alza.case.study@gmail.com,
PUBSUB_TOPIC=projects/gen-lang-client-0958735221/topics/
alza_case_study_email_inbox,
SEND_MODE=send" \
--set-secrets "GMAIL_CLIENT_ID=gmail-client-id:latest,
GMAIL_CLIENT_SECRET=gmail-client-secret:latest,
GMAIL_REFRESH_TOKEN=gmail-refresh-token:latest"
```

where

- **`GMAIL_CLIENT_ID`:** The public application identifier issued by Google Cloud OAuth 2.0, identifying the Alza support bot application to Google's authentication servers.
- **`GMAIL_CLIENT_SECRET`:** The confidential application password paired with the Client ID, used to authenticate the bot's authorisation requests and prevent identity impersonation.
- **`GMAIL_REFRESH_TOKEN`:** A long-lived OAuth credential that enables the headless Cloud Run service to autonomously request fresh, short-lived access tokens without requiring user re-authentication.

Additional environment variables and secrets were added manually later and managed by the GCP Secret Manager. For live deployment, the Pub/Sub OICD[6] audience and IAM policy bindings[7] needed to be set as well.

### 5.3 Final architecture and system flow diagram

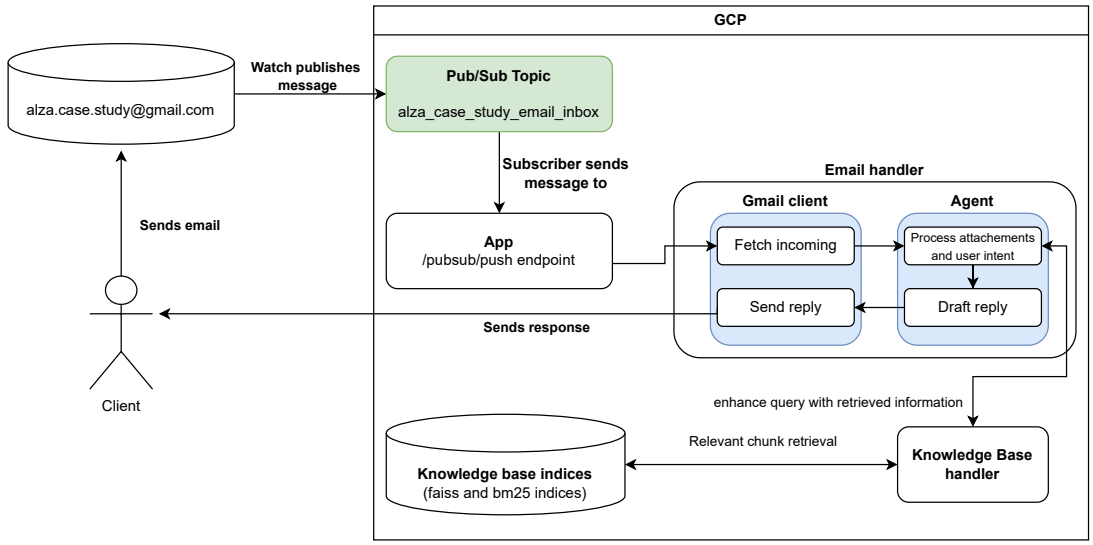



Figure 6: Final architecture with Pub/Sub notifications, email processing, LLM actions and RAG handling.

[6]OpenID Connect - an identity verification protocol built on top of OAuth 2.0.

[7]Identity and Access Management - linking a member (GCP service account) with a role (web service invoker) to perform a task (invoke the Cloud Run app).

# 6 Gaps and potential enhancements

This email-based AI chatbot was developed as a proof-of-concept, and therefore there are places for improvement and functionality expansion. Some of the current gaps include:

- **Security:**
  - **Lack of OICD validation with local deployment:** The local subscriber currently accepts all messages sent to the topic and does not check OICD authentication. The security risk is mitigated by the fact that the message only contains the monitored email address and a *history_id*, and the real content access only happens from the application side.
  - **Attachment spoofing:** Sophisticated payloads like malformed PDF objects containing JavaScript exploits, macro-embedded documents, Zip bombs or image/audio files containing hidden shellcode or exploit payloads can pass basic magic-byte validation.
- **Limited failure recovery:** If message processing fails (e.g., due to an API outage or unparseable payload), the app returns an HTTP 500 error to trigger Pub/Sub redelivery.
- **Static Knowledge Base Updates:** The knowledge base index files are pre-computed offline using *scripts/create_indices.py*. The running service cannot dynamically ingest new documents, update product manuals, or re-index content at runtime without restarting the container or triggering a manual rebuild pipeline.

Some of the potential updates and functionality enhancements include:

- **Fine-tuning the solution on a specific task:** the assignment did not state a specific role the agent should perform (client-facing customer support bot, internal document and knowledge base keeper, general "know-it-all" agent, etc.), which opens up many ways to polish system prompts and task descriptions.
- **Access to more tools:** based on the assignment, only one advanced feature was required to implement. Future versions of the application could allow the agent to access the internet for more up-to-date responses or consider an email thread to maintain the full conversation context with the end user, or any additional internal or external tools and MCP servers (Jira, Confluence, other communication channels (Teams, Slack, Outlook), etc.).
- **Better and bigger knowledge base maintenance:** The current setup is a good concept that's scalable to more documents, but not really as part of this application; the optimal approach would be to make a separate solution (a GCS vector database with a lightweight Cloud Run app, for example), that would handle ingestion of provided documents, their classification, parsing, chunking, indexing, storage and retrieval.

# 7 Appendix

## 7.1 Full codebase structure tree

Source code is available on GithHub:
https://github.com/xkoren10/alza_email_agent/tree/main

```
/   # Root Directory
├── main.py   # Entry point:  wires classes together and exposes Flask app
├── app/   # Core application runtime package
│   ├── config.py   # Settings loaded from environment variables
│   ├── models.py   # Data structures:  IncomingEmail, Classification, Reply, etc.
│   ├── agent.py   # SupportAgent:  Gemini API integration & orchestration
│   ├── knowledge_base.py   # KnowledgeBase:  hybrid search (Embeddings + BM25)
│   ├── embeddings.py   # Gemini embedding API client wrapper
│   ├── email_handler.py   # EmailHandler:  ingestion and lifecycle workflow
│   ├── gmail_client.py   # GmailClient:  Gmail REST API wrapper
│   ├── attachments.py   # Attachment security validation and magic-byte checks
│   └── server.py   # Flask HTTP endpoints:  webhooks and admin routes
├── knowledge_base/   # Data sources and precomputed indices
│   ├── raw_files/   # Raw input documents (PDF, XLSX, images)
│   ├── knowledge_chunks.json   # Intermediate parsed text chunks
│   └── vector_index/   # Precomputed indices (embeddings.npy, bm25.pkl)
├── scripts/   # Offline indexing and setup utility tools
│   ├── preprocess_files.py   # Converts raw files to knowledge_chunks.json
│   ├── create_indices.py   # Generates vector indices from chunks
│   └── get_gmail_token.py   # One-off OAuth refresh token generation
├── google/   # GCP helper utilities
│   ├── auth_second.py   # Auth helper for secondary test account
│   └── register_watch.py   # One-time manual setup of Gmail push watch
├── tests/   # Pytest unit test suite using standalone fakes
├── docs/   # Deployment documentation
│   └── DEPLOYMENT.md   # GCP step-by-step setup guide
├── Dockerfile   # Container build definition
└── .env.example   # Template for environment configuration
```

## 7.2 Running app screenshots

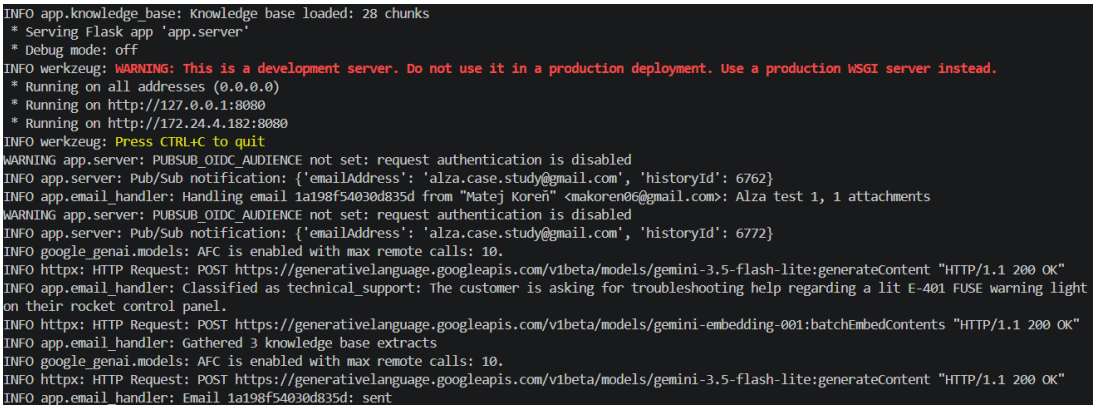



Figure 7: Screenshot of the app running locally, with incoming Pub/Sub notifications, message classification, knowledge base access and reply drafting.

## 7.3 Conversation examples

Gmail **Matej Koreň <makoren06@gmail.com>**

**Alza test 1**
Správ: (2)

**Matej Koreň** <makoren06@gmail.com> 1. novembra 2026 o 22:12
Komu: "alza.case.study@gmail.com" <alza.case.study@gmail.com>

Hello,

I have an issue with my rocket - there are some warning lights on my control panel, of which i am sending a picture in the attachements.
Can you tell me what's wring and how to fix it?
Thank you!
Matej



**Gemini_Generated_Image_ phvo2vphvo2vphvo.jpg**
594K

**John Doe** <alza.case.study@gmail.com> 1. novembra 2026 o 22:12
Komu: makoren06@gmail.com

Hello Matej,

Thank you for reaching out to Alza Support.

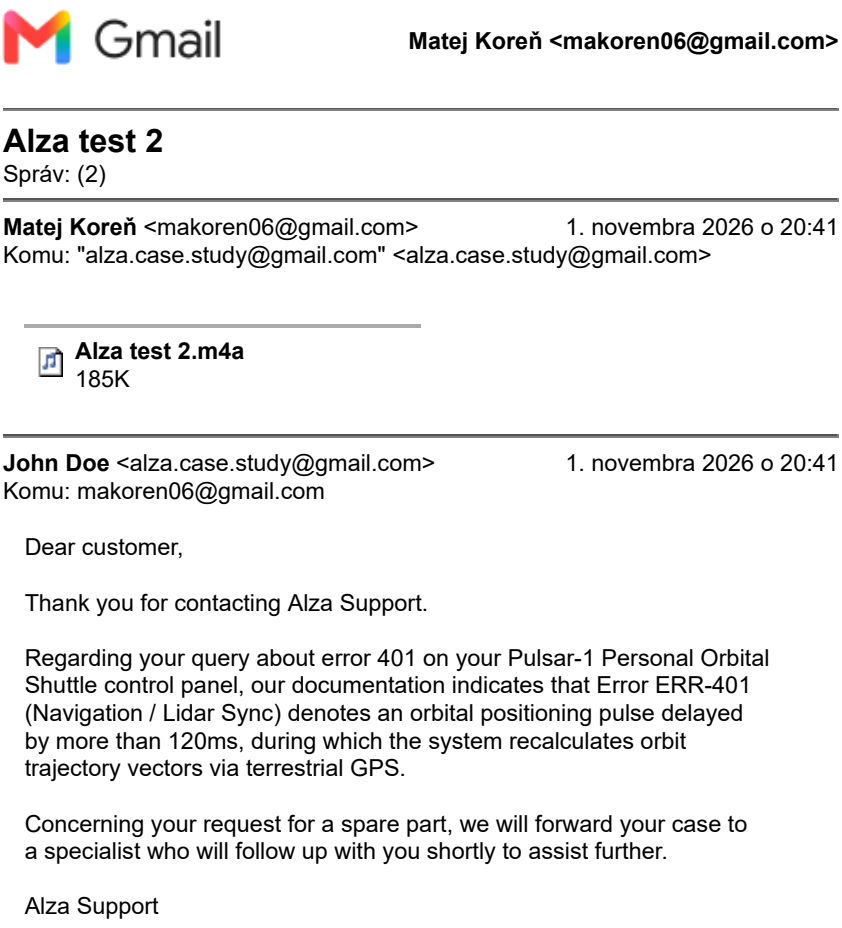
Regarding the warning light shown on your control panel for the E-401 FUSE, our records show that this corresponds to the Plasma Containment Fusion Fuse (500A) (Part ID: PLS-FUS-101), which is currently available in stock at our Prague Hub if a replacement is needed.

For further technical assistance regarding the troubleshooting and replacement procedure, a specialist will follow up with you shortly.

Best regards,

Alza Support

Figure 8: Example customer support request, showcasing the understanding of attachments (image of control panel with a control light on) and knowledge base retrieval (based on which error light is on, retrieve the relevant error description and see if the spare part that can fix the issue is in stock).



Gmail **Matej Koreň <makoren06@gmail.com>**

## Alza test 2

Správ: (2)

**Matej Koreň** <makoren06@gmail.com> 1. novembra 2026 o 20:41
Komu: "alza.case.study@gmail.com" <alza.case.study@gmail.com>

**Alza test 2.m4a**
185K

**John Doe** <alza.case.study@gmail.com> 1. novembra 2026 o 20:41
Komu: makoren06@gmail.com

Dear customer,

Thank you for contacting Alza Support.

Regarding your query about error 401 on your Pulsar-1 Personal Orbital Shuttle control panel, our documentation indicates that Error ERR-401 (Navigation / Lidar Sync) denotes an orbital positioning pulse delayed by more than 120ms, during which the system recalculates orbit trajectory vectors via terrestrial GPS.

Concerning your request for a spare part, we will forward your case to a specialist who will follow up with you shortly to assist further.

Alza Support

Figure 9: Example customer request, showcasing the ability to extract user intent from an attachment (voice recording describing the issue and error code, with a request for an available spare part) without any email body present.